

FS module CRUD operations using node.js

To perform operations on File System , we need to import fs module in the JavaScript file .

Importing fs module in the JavaScript file:

Syntax:

```
require("fs")
```

- Store the module in a variable for accessing further to perform some operations.

Example:

```
JS module.js > ...  
1  var fs = require("fs");
```

After Importing fs module , we are able to perform some operations in the file system.

Operations being performed on the file system are as follows:

1. Create a file
2. Read content from the file
3. Rename a file
4. Delete a file
5. Create a folder
6. Remove a folder.
7. Check if the file exists.

Let us perform the above operations in the file system using node.js

1.Create a file using fs module

Syntax:

```
fs.writeFile("filename" , "Content of file" , callBackFunction{ });
```

Example:

```
JS module.js > ...  
1  var fs = require("fs");  
2  
3  fs.writeFile("read.txt","This is a dummy read me file" , function(error){  
4    if(error){  
5      console.log("Failed to create a file");  
6    }else{  
7      console.log("File created successfully");  
8    }  
9  })
```

If we run this file in the terminal it displays the File created successfully and we can find the read.txt file in the same folder where we created the JavaScript file.

2.Read the content from the file

Syntax:

```
fs.readFile("filename" , "utf-8" , callBackFunction{ });
```

Example:

```
JS module.js > ...
1  var fs = require("fs");
2
3  fs.readFile("read.txt","utf-8" , function(error,data){
4      if(error){
5          console.log("Failed to rename the file");
6      }else{
7          console.log(data);
8      }
9  })
```

If we run the above file we can see the content of the file in the console.

Note: Here we are using "utf-8" to keep the content of the file as same without changing it to hexadecimal format.

3.Rename a file using fs module

Syntax:

```
fs.rename("filename" , "newName" , callBackFunction{ });
```

Example:

```
JS module.js > ...
1  var fs = require("fs");
2
3  fs.reaname("read.txt","document" , function(error){
4      if(error){
5          console.log("Failed to rename the file");
6      }else{
7          console.log("File renamed sucessfully");
8      }
9  })
```

If we run the above file we can see that the File is renamed in the console and we can find the renamed file in the same folder where we created JavaScript file.

4.Delete a file using fs module

Syntax:

```
fs.unlink("filename" , callBackFunction{ });
```

Example:

```
JS module.js > ...
1  var fs = require("fs");
2
3  fs.unlink("read.txt" , function(error){
4    if(error){
5      console.log("Failed to delete the file");
6    }else{
7      console.log("File deleted successfully");
8    }
9  })
```

If we run the file with the correct file name then we are able to see that the file deleted line is displayed in the console and the file would be deleted.

5.Create a folder using fs module

Syntax:

```
fs.mkdir("folderName" , callBackFunction{ });
```

Example:

```
JS module.js > ...
1  var fs = require("fs");
2
3  fs.mkdir("NewFolder" , function(error){
4    if(error){
5      console.log("Failed to create the folder");
6    }else{
7      console.log("Folder created successfully");
8    }
9  })
```

If we run this file we are able to see that the folder is created in the same folder where JavaScript file is created.

6.Remove a folder using fs module

Syntax:

```
fs.rmdir("folderName" , callBackFunction{ });
```

Example:

```
JS module.js > ...
1  var fs = require("fs");
2
3  fs.rmdir("NewFolder" , function(error){
4      if(error){
5          console.log("Failed to remove the folder");
6      }else{
7          console.log("Folder removed successfully");
8      }
9  })
```

If we run the file we are able to find that the folder is deleted.

7.Check if the given file exists using fs module

Syntax:

```
fs.existsSync("fileName" , callBackFunction{ });
```

Example:

```
JS module.js > ...
1  var fs = require("fs");
2
3  var result = fs.existsSync("read.txt");
4
5  console.log(result);
```

Note: Here `existsSync()` return the boolean value and returns true or false . So we are storing it in the variable and printing the variable .

If we run the file we can find **true** is displayed in the console if the file exists ,if not **false** is displayed.

Similarly there are many operations which can be performed using the fs module. In our learning path of NodeJS let's explore few more operations later.